RICKSPLORADOR DEL MORTYVERSO

Rick&Morty API

Aplicación web interactiva
JavaScript + POO + DOM + fetch



Markel · Eder · Ruth· Guillermo · Eneko

Octubre 2026

INDEX

01	**Objetivo y MVP**	Problema, personas y alcance	02
02	**Solución UX/UI**	Interfaz, responsive y accesibilidad	07
03	**Desarrollo técnico**	Arquitectura Mermaid, POO, DOM, API	12
04	**Trabajo en equipo**	Roles, Git y Gantt	18
05	**Validación y cierre**	Pruebas, incidencias, conclusiones	22
06	**Recursos**	Webgrafía	26

01 OBJETIVO Y MVP

Problema, personas y alcance.

Ricksplorador del Mortyverso es una aplicación web interactiva basada en la API de Rick and Morty que permite explorar los personajes, episodios y localizaciones del multiverso de la serie.

MVP

- **Visualización de datos**
  Personajes, episodios y localizaciones.
- **Búsqueda y filtrado**
  Por nombre y otros criterios.
- **Favoritos**
  Guardar y eliminar personajes y episodios.
- **Paginación**
  Navegación entre resultados.



Markel · Eder · Ruth · Guillermo ·Eneko

¿Qué problema queremos resolver?

Una forma sencilla de explorar el universo de Rick y Morty.

NECESIDAD

La información de personajes, episodios y lugares está repartida entre muchos contenidos. Encontrarla requiere varias consultas.

01 / PROBLEMA

PROPUESTA

Una sola web con catálogos, fichas, navegación por secciones y favoritos guardados en el navegador.

02 / SOLUCIÓN

BENEFICIO

Consulta visual, búsqueda y retorno rápido a los elementos que cada persona quiere conservar.

03 / VALOR

Recorrido principal del usuario

Una historia de uso para explicar el MVP sin entrar todavía en el código.

"Quiero encontrar un episodio o personaje y guardarlo para consultarlo después."

01	02	03	04	05
ENTRAR	**EXPLORAR**	**CONSULTAR**	**GUARDAR**	**RECUPERAR**
Inicio y menú	Elegir catálogo	Ver tarjeta o ficha	Pulsar corazón	Abrir Favoritos

El contenido cambia sin recargar toda la web; los favoritos permanecen mediante localStorage.

Cuatro áreas, una misma experiencia

Mapa funcional de la aplicación.

01

PERSONAJES

Explorar, filtrar y consultar

Nombre, estado, especie y localización

02

LOCALIZACIONES

Explorar lugares y dimensiones

Dimensiones y residentes

03

EPISODIOS

Recorrer capítulos

Temporada, episodio y fecha de estreno

04

FAVORITOS

Recuperar lo guardado

Personajes, localizaciones y episodios

Alcance del MVP

Diferenciamos lo implementado de lo que hay que dejar para mas adelante.

01 IMPLEMENTADO

Consulta a Rick and Morty API mediante fetch.

Clases para personajes, episodios y lugares.

Navegación entre pantallas y fichas.

Filtros de búsqueda en personajes.

Favoritos de personajes, localizaciones y episodios.

02 PENDIENTE DE MEJORA

División de episodios por temporada.

Incluir enlace para verlo.

Incluir filtros en localizaciones y episodios.

Cambiar a modo oscuro.

Autentificación.

02
SOLUCIÓN UX/UI

Diseño que acompaña la exploración.

Identidad inspirada en el multiverso.

UX/UI

Diseño visual
Interfaz inspirada en el universo de la serie.

Experiencia responsive
Adaptación a diferentes dispositivos.

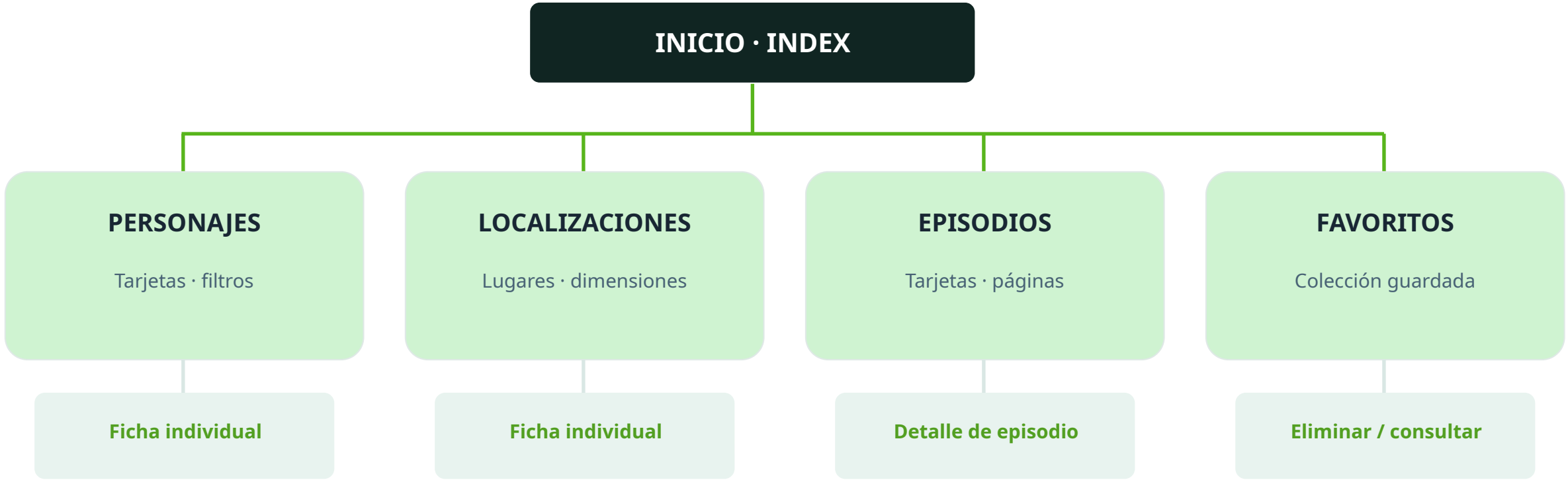
Navegación intuitiva
Exploración fácil y fluida.

Estilo coherente
Colores, tipografías y elementos unificados.



Arquitectura de información

Desde Inicio se accede a catálogos y a la colección de favoritos.



Identidad visual

Guía visual para unificar las secciones.

PALETA

BLANCO

Fondos



CIAN

Acciones



TURQUESA

Navegación



CARBÓN

Textos

MENTA

Apoyos

TIPOGRAFÍA

Arial · lectura clara

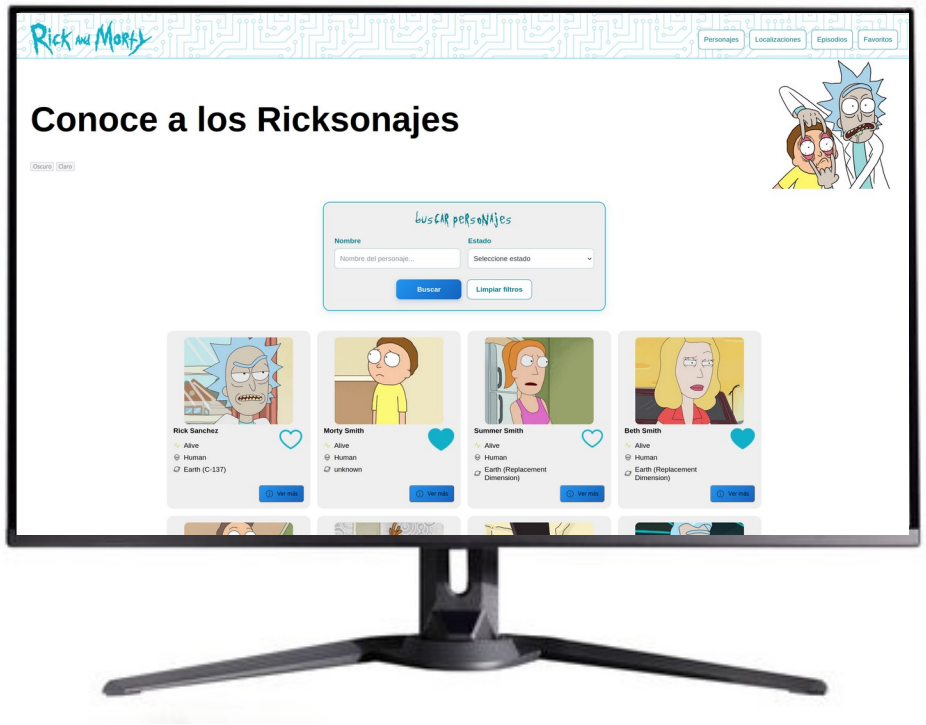
IDENTIDAD

Get Schwifty · acento temático

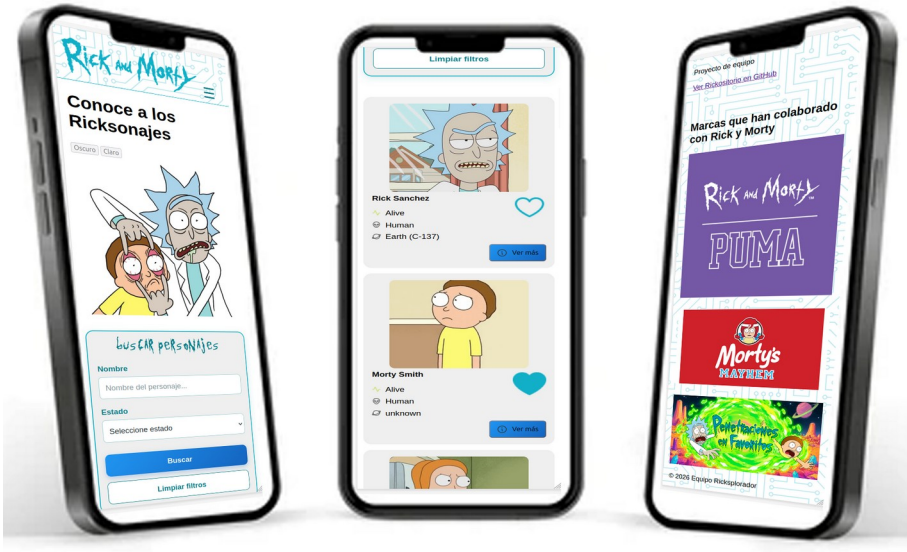
Diseño responsive

Distribución adaptada al ancho disponible.

Versión ESCRITORIO



Versión MÓVIL



Patrones de interfaz reutilizables

Las tarjetas mantienen la misma identidad visual, aunque sus datos sean diferentes.

PERSONAJES



LOCALIZACIONES



EPISODIOS



03 DESARROLLO TÉCNICO

De la API a la pantalla.

Clases para estructurar los datos, eventos para interactuar y localStorage para conservar favoritos.

- `fetch()`
- `response.json()`
- `new Personaje()`
- `createElement()`
- `localStorage`



Arquitectura y flujo de datos

Esquema de cómo se conectan las partes de la aplicación.



Se consulta la API para obtener datos. El navegador construye las tarjetas, gestiona los clics y guarda preferencias.

Modelo orientado a objetos

Clases reales identificadas en el proyecto: Personaje, Lugar y Episodio.

Personaje

PROPIEDADES

id · name · status
species · origin · image

MÉTODOS

isAlive()
filterByName()
filtrerByStatus()

Lugar

PROPIEDADES

id · name · type
dimension · residents

MÉTODOS

countResidents()

Episodio

PROPIEDADES

id · name · estreno
temporada · episodio · personajes

MÉTODOS

constructor(data)

Peticiones a una API externa

La información se obtiene mediante funciones asíncronas y se transforma en objetos.

EXTRACTO REAL · js/main.js

```
export async function getPersonajes(numberPage = 1) {
    const API_URL = "https://rickandmortyapi.com/api/character";

    const url = numberPage === 1 ?
        API_URL
        : `${API_URL}/?page=${numberPage}`;

    const response = await fetch(url);

    if (!response.ok) {
        throw new Error('Error al obtener los datos del personaje');
    }
    const data = await response.json();

    if (!data.results || data.results.length === 0) {
        throw new Error("No hay datos")
    }
```

01 **FETCH**

Solicita datos a la API.

02 **AWAIT / JSON**

Espera y lee la respuesta.

03 **MAP + NEW**

Crea objetos de clase.

API pública: /character · /location · /episode

DOM: de los datos a las tarjetas

JavaScript crea elementos y los incorpora a la página cuando se necesitan.

EJEMPLO DE REUTILIZACIÓN

```
// Creo una tarjeta por cada personaje guardado y la añado a la página.
personajesFavoritos.forEach(function (personaje) {
    const tarjeta = crearTarjetaPersonaje(personaje, true);
    contenedorPersonajes.appendChild(tarjeta);
});

// Creo una tarjeta por cada lugar guardado y la añado a la página.
lugaresFavoritos.forEach(function (lugar) {
    const tarjeta = crearTarjetaLugar(lugar, true);
    contenedorLugares.appendChild(tarjeta);
});

//Crear tarjetas para los episodios favoritos.
episodiosFavoritos.forEach(function (episodio) {

    const tarjeta = crearTarjetaEpisodioFav(episodio, true);

    contenedorEpisodios.appendChild(tarjeta);

});
```

01 REUTILIZAR

Una función construye la misma tarjeta para dos pantallas.

02 RENDELIZAR

appendChild inserta la tarjeta en la página.

03 ACTUALIZAR

El evento modifica la lista y elimina la tarjeta visible.

Ventaja: menos código duplicado y cambios más fáciles de mantener.

Favoritos: persistencia local

Los elementos seleccionados se guardan en el navegador y pueden recuperarse en otra pantalla.

01
MARCAR
El usuario pulsa el corazón.

02
GUARDAR
setItem + JSON.stringify

03
LEER
getItem + JSON.parse

04
MOSTRAR
forEach y tarjeta reutilizable

Claves de almacenamiento utilizadas	Qué guarda
`PersonajesFavorito`	Los personajes favoritos.
`episodiosFavorito`	Los episodios favoritos.
`LugaresFavorito`	Las localizaciones favoritas.

Se consulta la API para obtener datos. El navegador construye las tarjetas, gestiona los clics y guarda preferencias.

04

TRABAJO EN EQUIPO

Cinco personas, un repositorio.

La coordinación y las ramas permiten integrar funcionalidades desarrolladas en paralelo.

EQUIPO

- (ED) **Localizaciones**
- (EN) **Episodios**
- (GU) **Personajes**
- (MA) **API y coordinación**
- (RU) **Inicio y Favoritos**



Reparto de responsabilidades

Distribución del trabajo.

M	**Markel**	**API · COORDINACIÓN · PERSONAJES**	Conexión de datos, coordinación del equipo y apoyo a personajes.
E	**Eder**	**LOCALIZACIONES**	Vista, estilos y funcionalidad de Localizaciones y su pagina individual.
R	**Ruth**	**INICIO · FAVORITOS · MEMORIA**	Vista, estilos y funcionalidad de Inicio y Favoritos y diseño y maquetación de la memoria.
G	**Guillermo**	**PERSONAJES · README · MEMES**	Lista dinámica de personajes, Filtrado de personajes, Paginación de personajes, README y Memes
E	**Eneko**	**EPISODIOS**	Vista, estilos y funcionalidad de Episodios y su pagina individual.

Control de versiones

Esquema del flujo de trabajo con Git y GitHub.

RAMA DE FUNCIONALIDAD

Cada persona desarrolla su apartado.

`feature/...`

COMMIT LOCAL

Se guardan cambios con un mensaje claro.

`git commit`

PUSH + PULL REQUEST

Se comparte el trabajo y se solicita revisión.

`GitHub`

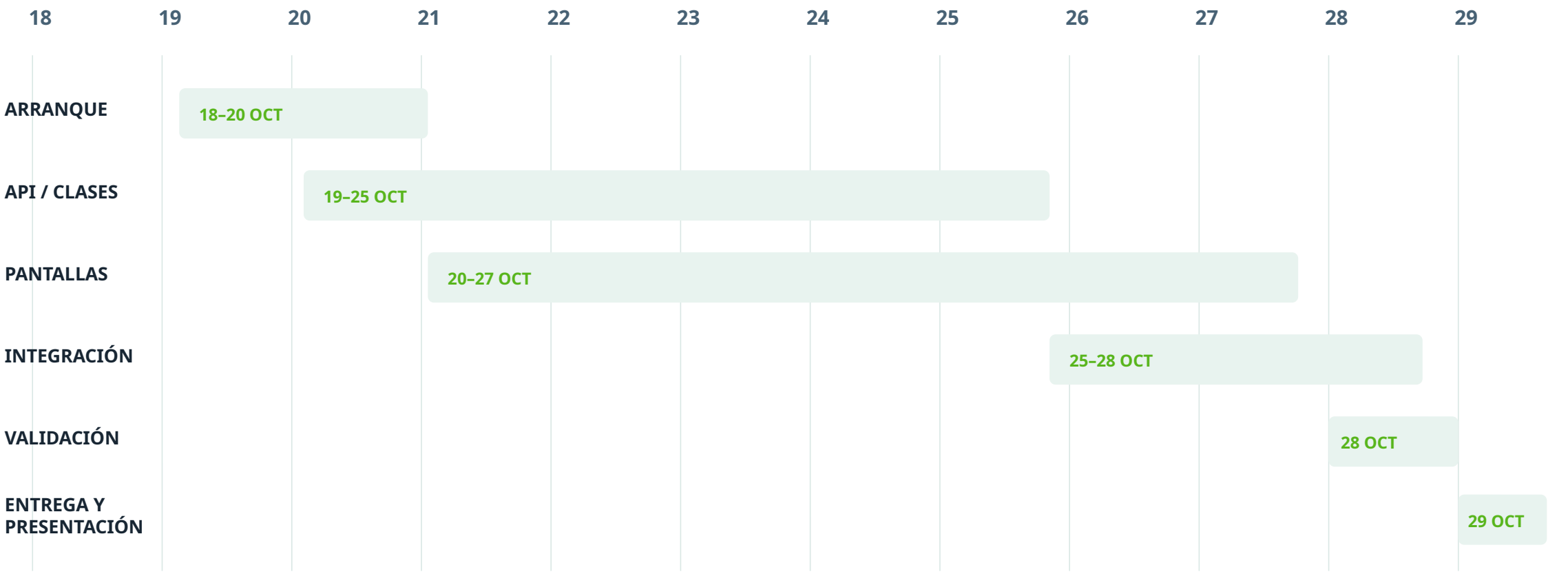
INTEGRACIÓN EN DEV

Se unen las funcionalidades y se prueban.

`dev`

Cronograma de Gantt

Periodo oficial: del 18 al 29 de octubre de 2026.



05 VALIDACIÓN Y CIERRE

Verificado antes de entregar.

Pruebas funcionales, revisión técnica y una memoria que describa exactamente lo realizado.

VALIDACIÓN

Funciona
Todas las funcionalidades operativas.

Se entiende
Código claro y bien organizado.

Se documenta
Memoria completa y estructurada.



Matriz de pruebas

Lista de comprobaciones.

FUNCIÓN	CÓMO COMPROBAR	ESTADO
Carga de la API	Abrir cada catálogo y cambiar de página.	✓
Filtros de personajes	Buscar por nombre y cambiar estado.	✓
Favoritos de episodios	Añadir, visitar Favoritos y eliminar.	✓
Favoritos de personajes	Añadir, recargar y desmarcar.	✓
Localizaciones favoritas	Comprobar guardado y aparición en Favoritos.	✓
Responsive / navegación	Probar menú y tarjetas en móvil.	✓

Decisiones y dificultades técnicas

Ejemplos reales.

REUTILIZACIÓN

RETO Se detectó código repetido al crear tarjetas de personajes.

RESPUESTA Se separó la construcción en tarjetaPersonaje.js.

COHERENCIA VISUAL

RETO Las tarjetas de episodios en Favoritos no mantenían las mismas medidas.

RESPUESTA Se ajustó la estructura y el CSS tomando Episodios como referencia.

INTEGRACIÓN

RETO Cada integrante trabajaba en archivos y ramas diferentes.

RESPUESTA Se emplearon Git, commits y fusiones para coordinar cambios.

Conclusiones y entregables

Resultados definitivos del equipo.

QUÉ HEMOS APRENDIDO

- Consumir datos externos mediante funciones asíncronas.
- Trabajar con clases, objetos y eventos del DOM.
- Guardar datos en el navegador con localStorage.
- Coordinar cambios en distintas ramas de Git.
- Revisar la coherencia visual entre módulos.

ENTREGA

01 · REPOSITORIO

GitHub · Rickositorio

02 · README.md

Actualizar instalación y uso

03 · MEMORIA

PDF

04 · EVIDENCIAS

Capturas + pruebas

06 RECURSOS

Webgrafía, referencias y programas utilizados

Fuentes consultadas, herramientas empleadas y referencias que han hecho posible este proyecto.

RECURSOS

- **Webgrafía**
  Enlaces y recursos consultados.
- **Referencias**
  Inspiración y documentación.
- **Programas utilizados**
  Herramientas y tecnologías.



Webgrafía y referencias

The Rick and Morty API. (s. f.). Documentation. Documentación oficial de la API utilizada para consultar y recuperar información sobre personajes, episodios y localizaciones. https://rickandmortyapi.com/

BBK y The Bridge. (2026). Bootcamp de Desarrollo Web Full Stack con IA. Materiales didácticos, ejercicios, ejemplos de código y recursos proporcionados durante la formación. https://thebridge.tech/campus-online/

Software y herramientas utilizadas

Visual Studio Code. Editor de código utilizado para desarrollar y organizar los archivos del proyecto.

Figma. Herramienta de diseño utilizada para trabajar la interfaz y su organización visual. https://www.figma.com/design/

Git. Sistema de control de versiones utilizado para registrar y gestionar los cambios realizados.

GitHub. Plataforma utilizada para alojar el repositorio, trabajar mediante ramas y coordinar la integración del código del equipo.

LibreOffice. Herramienta utilizada para elaborar y maquetar la memoria del proyecto.

ChatGPT (OpenAI). Herramienta de IA utilizada como apoyo en el aprendizaje, la consulta de dudas técnicas y la elaboración de imágenes.

Live Server. Extensión de Visual Studio Code utilizada para visualizar y comprobar el funcionamiento de la aplicación durante el desarrollo.

Tecnologías de desarrollo

HTML5: estructura y contenido de las páginas.

JavaScript: programación de la lógica y las funcionalidades interactivas.

Programación orientada a objetos (POO): organización de los datos mediante clases.

DOM: creación y modificación dinámica de elementos HTML.

Fetch API: consulta y recuperación de datos externos.

localStorage: almacenamiento de los favoritos en el navegador.

Tipografía

Font Meme. (s. f.). Get Schwifty Font. Tipografía diseñada por jonizaak, utilizada para reforzar la identidad visual de Rickositorio. https://fontmeme.com/fonts/get-schwifty-font/

Créditos y propiedad intelectual

Ricksplorador del Mortyverso es un proyecto académico desarrollado con fines educativos y sin finalidad comercial por Markel, Eder, Ruth, Guillermo y Eneko en el marco del bootcamp de desarrollo web Full Stack con IA.

El código y los desarrollos originales de Rickositorio son fruto del trabajo conjunto de sus cinco integrantes. Los personajes, imágenes y elementos de Rick y Morty pertenecen a sus respectivos titulares y se utilizan únicamente con fines educativos y sin ánimo de lucro.

RICKSPLORADOR DEL MORTYVERSO

Memoria del proyecto

Rick&Morty API

Aplicación web interactiva
JavaScript + POO + DOM + fetch

Markel · Eder · Ruth · Guillermo · Eneko

Octubre 2026



Ver Rickositorio
en GitHub